

Predictive Insight Engine

Supervised Learning: Theory Concepts & Definitions

House Price Prediction using Regression Techniques

1. About this Document

This document explains the theory behind every technique used in the **Predictive Insight Engine** notebook. Each concept has a simple **definition**, the **formula** (where needed), and how it appeared in *our own results*, so the theory and the practical work connect.

Problem: As a Junior Data Scientist at a real-estate firm, predict the **house price (house_price_inr)** from features such as area, bedrooms, bathrooms, location score, age, distance from city, lot size, garage, pool and years since renovation.

Item	Value
Total records	4,200 houses, no missing values
Train / Test split	3,360 training rows (80%) and 840 test rows (20%)
Average house price	about ₹2.36 crore (₹23.6 million)
Target variable (y)	house_price_inr
Features (X)	area_sqft, bedrooms, bathrooms, location_score, age_years, distance_city_km, lot_size_sqft, has_garage, has_pool, renovation_years_ago

2. Supervised Learning

DEFINITION Supervised learning is a type of machine learning where the model learns from **labelled data**: every input (features, X) comes with the correct output (target, y). The model learns the mapping $y = f(X)$ and then predicts y for new, unseen inputs.

It is called *supervised* because the known answers act like a teacher that corrects the model during training. There are two main problem types:

- **Regression:** predict a continuous number (price, temperature).
- **Classification:** predict a category (spam / not spam, disease / no disease).

Common algorithms: Linear Regression, Logistic Regression, Decision Tree, Random Forest, KNN, SVM.

EXAMPLE Features = area, bedrooms, location score. Label = price. After learning from 3,360 known houses, the model predicts the price of a new house.

3. Regression vs Classification

Point	Regression	Classification
Output type	Continuous numeric value	Discrete class / label
Example	House price = ₹2.4 crore	Email = Spam or Not Spam
Algorithms	Linear, Polynomial, Ridge, Lasso	Logistic, KNN, SVM, Decision Tree
Evaluation metrics	MSE, MAE, RMSE, R^2	Accuracy, Precision, Recall, F1
Our project	Yes, price is a number	No

4. Dataset Concepts

4.1 Independent and Dependent Variables

DEFINITION Independent variables (features, X) are the inputs used to make a prediction. The dependent variable (target, y) is the value we want to predict; it depends on the independent variables.

In our project the target is **house_price_inr**. The column *house_id* is only an identifier, so it is dropped and not used as a feature.

4.2 Train-Test Split

DEFINITION Train-test split divides data into a **training set** (used to learn the parameters) and a **testing set** (kept hidden and used only to check performance on unseen data).

We used an **80% / 20% split** (3,360 / 840 rows) with `random_state = 42` so the result is reproducible. Testing on unseen data tells us whether the model truly generalises or just memorised the training data.

4.3 Correlation

DEFINITION **Correlation (r)** measures the strength and direction of a linear relationship between two variables, from -1 (perfect negative) to $+1$ (perfect positive). 0 means no linear relationship.

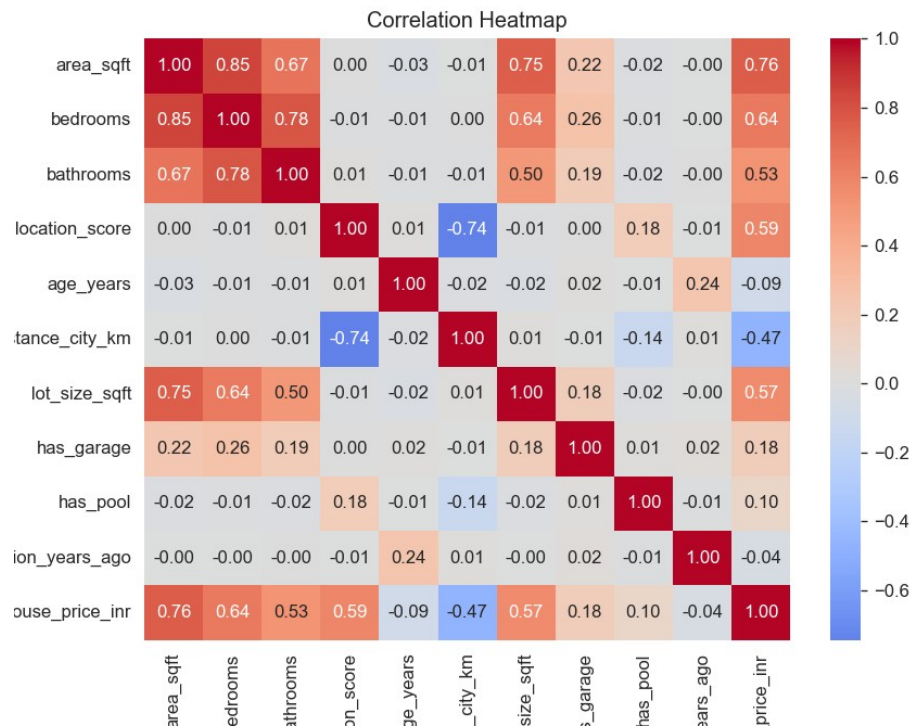


Figure 1: Correlation heatmap of all features and the target

OBSERVATION **area_sqft (0.76)** has the strongest link with price, then **bedrooms (0.64)**, **location_score (0.59)**, **lot_size_sqft (0.57)** and **bathrooms (0.53)**. **distance_city_km (-0.47)** is negative: farther houses are cheaper. **age_years (-0.09)** and **renovation_years_ago (-0.04)** are weak. Features are also correlated with each other: **area-bedrooms 0.85**, **bedrooms-bathrooms 0.78**, **area-lot size 0.75**, **location-distance -0.74**. This is called multicollinearity (see Section 8).

5. Simple Linear Regression

DEFINITION Simple Linear Regression models the relationship between **one independent variable (x)** and one dependent variable (y) using a straight line. The best line is found using the **Least Squares method**, which minimises the sum of squared differences between actual and predicted values.

$$y = \beta_0 + \beta_1 \cdot x$$

- **β_0 (intercept):** predicted y when x = 0; it is where the line crosses the y-axis.
- **β_1 (slope):** change in y for a one-unit increase in x.
- **Residual (error):** actual value – predicted value ($e = y - \hat{y}$).

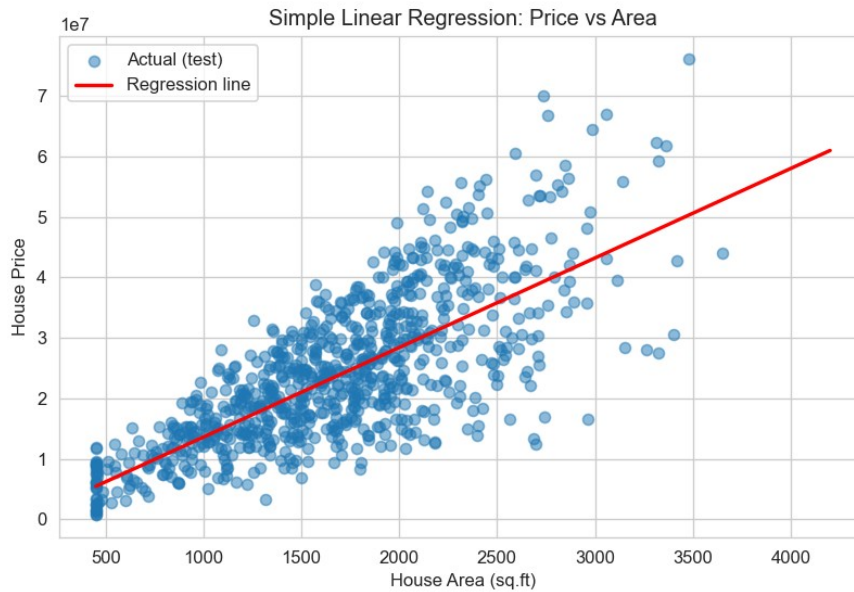


Figure 2: Simple Linear Regression, price vs area (test data)

OBSERVATION Fitted equation: **Price = -1,163,519 + 14,788.31 × Area**. The slope means every extra square foot adds about ₹14,788 to the price. The intercept is a mathematical baseline only, because a house with 0 sq.ft does not exist. The points spread widely around the line, so area alone explains only about 56% of price variation ($R^2 = 0.563$).

6. Assumptions of Linear Regression

Linear regression gives reliable results only when these assumptions hold approximately:

Assumption	Meaning	How to check	Result in our project
1. Linearity	X and y have a straight-line relationship	Residuals vs fitted plot (random scatter around 0)	Scatter is centred on 0 with no strong curve, so the relation is roughly linear
2. Independence	Errors are not correlated with each other	Durbin-Watson test (≈ 2 is good)	Durbin-Watson = 1.98, so errors are independent
3. Homoscedasticity	Error variance is constant for all predictions	Residuals vs fitted (no funnel shape)	A mild funnel: errors grow for expensive houses (slight violation)
4. Normality of errors	Residuals follow a bell-shaped curve	Histogram, Q-Q plot, Shapiro-Wilk test	Bell-shaped histogram, Q-Q points close to the line (small tail deviation)
5. No multicollinearity	Features are not highly correlated with each other	Correlation matrix, VIF	Present between area, bedrooms, bathrooms, lot size (see Figure 1)

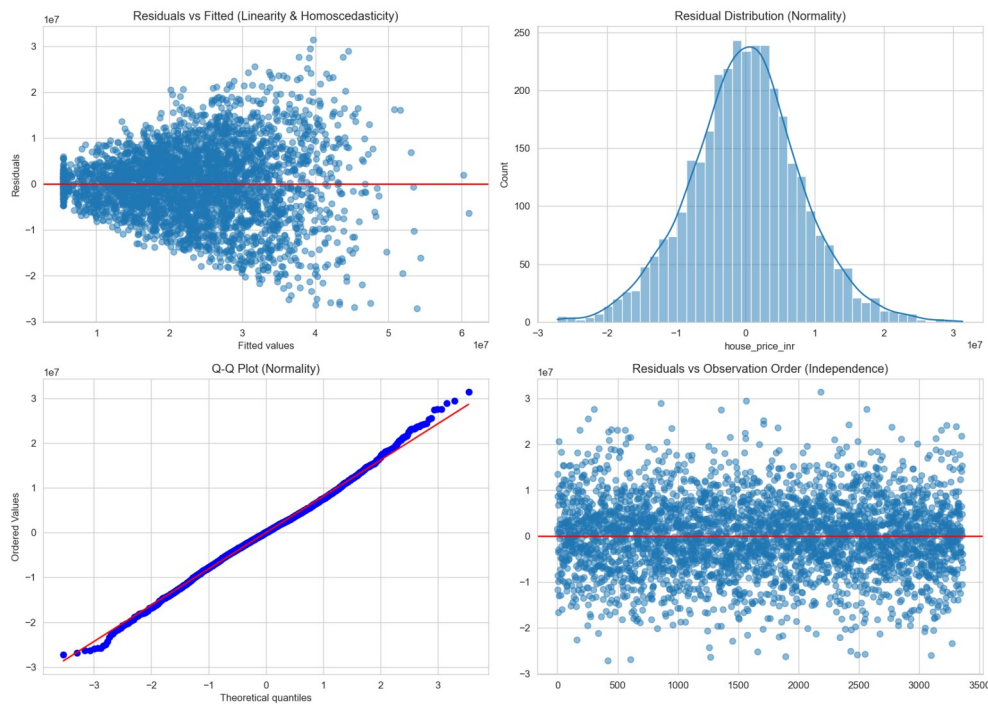


Figure 3: Residual diagnostics (residual vs fitted, histogram, Q-Q plot, residual order)

OBSERVATION The Shapiro-Wilk p-value prints as 0.0, which formally rejects perfect normality. With thousands of rows this test becomes extremely strict and flags even tiny deviations, so the histogram and Q-Q plot (which look nearly normal) are more useful. The widening spread in the first plot is a mild sign of heteroscedasticity; a log transform of price or a weighted model could reduce it.

7. Model Evaluation Metrics

Metrics convert the difference between actual and predicted values into a single number so that models can be compared. Here n = number of samples, y_i = actual, $\hat{y}_i$ = predicted, p = number of features.

Metric	Formula	Meaning and interpretation
MSE (Mean Squared Error)	$(1/n) \sum (y_i - \hat{y}_i)^2$	Average squared error; punishes large errors heavily. Unit is price ² , so it is hard to read directly. Lower is better.
MAE (Mean Absolute Error)	$(1/n) \sum y_i - \hat{y}_i $	Average size of the error in rupees; easy to understand and less affected by outliers.
RMSE (Root MSE)	$\sqrt{\text{MSE}}$	Typical error in rupees. It is larger than MAE when some big mistakes exist.
R² Score	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	Share of price variation explained by the model (0 to 1). 0.92 means 92% is explained.
Adjusted R²	$1 - (1 - R^2)(n - 1) / (n - p - 1)$	R ² corrected for the number of features. It rises only if a new feature truly helps, so it is fair for comparing models of different sizes.

Results of Simple Linear Regression (test set)

MSE	MAE	RMSE	R ²	Adj. R ²
6.70×10^{13}	₹62.9 lakh	₹81.8 lakh	0.563	0.562

OBSERVATION On average the Simple LR prediction is off by about **₹63 lakh** (MAE), and the typical error is **₹82 lakh** (RMSE), which is large compared with the average price of ₹2.36 crore. $R^2 = 0.563$ says one feature explains only 56% of the variation. The model is too simple, so we add more features.

8. Multiple Linear Regression

DEFINITION Multiple Linear Regression predicts y using **two or more independent variables**. Each coefficient shows the effect of its feature on y while the other features are held constant.

$$y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_px_p$$

Multicollinearity

DEFINITION **Multicollinearity** happens when independent variables are strongly correlated with each other. It makes individual coefficients unstable and hard to interpret, although overall prediction can still be good.

Learned coefficients (all 10 features)

Feature	Coefficient	Meaning (other features constant)
area_sqft	+13,796	+₹13,796 per extra sq.ft
bedrooms	+1,97,643	+₹1.98 lakh per extra bedroom
bathrooms	+1,85,945	+₹1.86 lakh per extra bathroom
location_score	+30,73,174	+₹30.7 lakh per 1-point better location (largest effect)
age_years	-65,609	-₹0.66 lakh per year of age
distance_city_km	-98,965	-₹0.99 lakh per extra km from city
lot_size_sqft	+103.5	+₹104 per extra sq.ft of plot
has_garage	+93,924	Garage adds about ₹0.94 lakh
has_pool	+4,55,290	Pool adds about ₹4.55 lakh
renovation_years_ago	-21,436	-₹0.21 lakh for each year since renovation

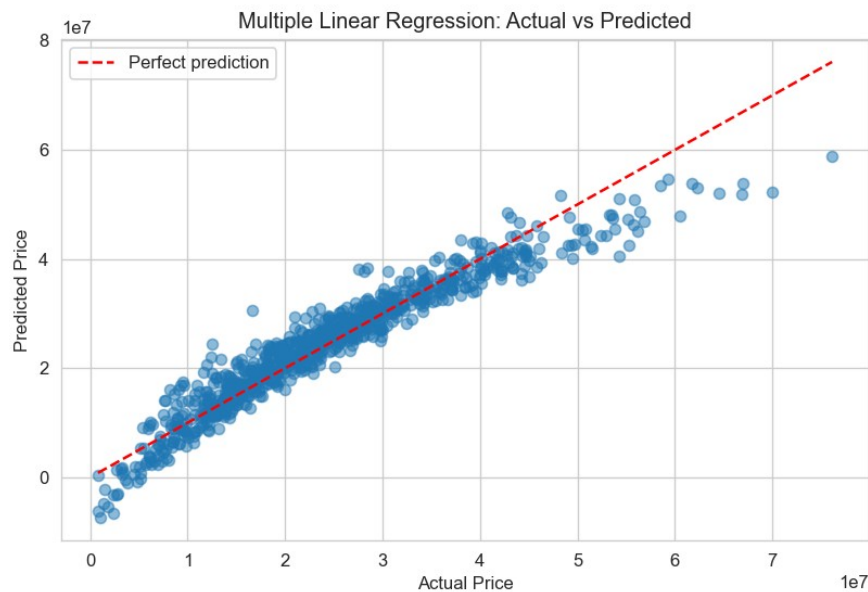


Figure 4: Multiple Linear Regression, actual vs predicted price

OBSERVATION Points lie close to the red perfect-prediction line, so the model tracks price well. It under-predicts the most expensive houses (points fall below the line above ₹5 crore) and produces a few negative predictions for very cheap houses, which shows a straight-line model cannot fully capture very high or very low prices.

Simple vs Multiple Linear Regression

Metric	Simple LR	Multiple LR	Change
RMSE	₹81.8 lakh	₹35.5 lakh	56% lower
MAE	₹62.9 lakh	₹26.0 lakh	59% lower
R ²	0.563	0.918	+0.355
Adjusted R ²	0.562	0.917	+0.355

Why performance improves: price depends on many factors, not only area. Adding location, bedrooms, distance, lot size, etc. reduces **bias** and explains more variance. Adjusted R² also rises (0.562 → 0.917), which proves the new features add real information. Performance can degrade if irrelevant or highly correlated features are added, or when data is very small.

9. Polynomial Regression

DEFINITION Polynomial Regression models a **non-linear relationship** by adding powers of the features (x^2 , x^3 , ...) and then fitting a linear regression on these new features. It is still linear in the coefficients, but the fitted curve can bend.

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_k x^k$$

Degree 2 gives one bend (parabola), degree 3 gives up to two bends. With many features, polynomial terms also include interactions (for example area × location_score): with 10 features,

degree 2 creates **65** features and degree 3 creates **285** features. Because higher powers create very large numbers, features are **standardised** first.

Model comparison (test set)

Model	MSE ($\times 10^{12}$)	MAE (₹)	RMSE (₹)	R ²	Adj. R ²
Simple LR	66.99	6,294,594	8,184,697	0.563	0.562
Multiple LR	12.59	2,604,991	3,548,650	0.918	0.917
Polynomial (deg 2)	5.14	1,656,923	2,267,045	0.966	0.964
Polynomial (deg 3)	5.41	1,707,651	2,326,734	0.965	0.946

OBSERVATION Degree 2 is the best: RMSE falls from ₹35.5 lakh (Multiple LR) to ₹22.7 lakh, and R² rises to 0.966. Degree 3 is slightly worse (RMSE ₹23.3 lakh) and its Adjusted R² drops to 0.946 because 285 features are too many. It means the extra complexity is not worth it. When only area is used, the degree 1, 2 and 3 curves almost overlap and R² stays near 0.56, so the area-price relation is almost linear; the improvement comes from feature interactions (for example area with location), not from bending the area line.

10. Overfitting and Underfitting

DEFINITION Underfitting: the model is too simple to capture the pattern, so error is high on both training and test data.

DEFINITION Overfitting: the model is too complex and learns noise in the training data, so training error is very low but test error is high.

	Underfitting	Good fit	Overfitting
Training error	High	Low	Very low
Testing error	High	Low (close to train)	High
Bias / Variance	High bias, low variance	Balanced	Low bias, high variance
Cause	Model too simple, too few features	Right complexity	Too complex, too many features, little data
Fix	Add features or complexity	Keep it	Simpler model, more data, regularisation (Ridge/Lasso), cross-validation

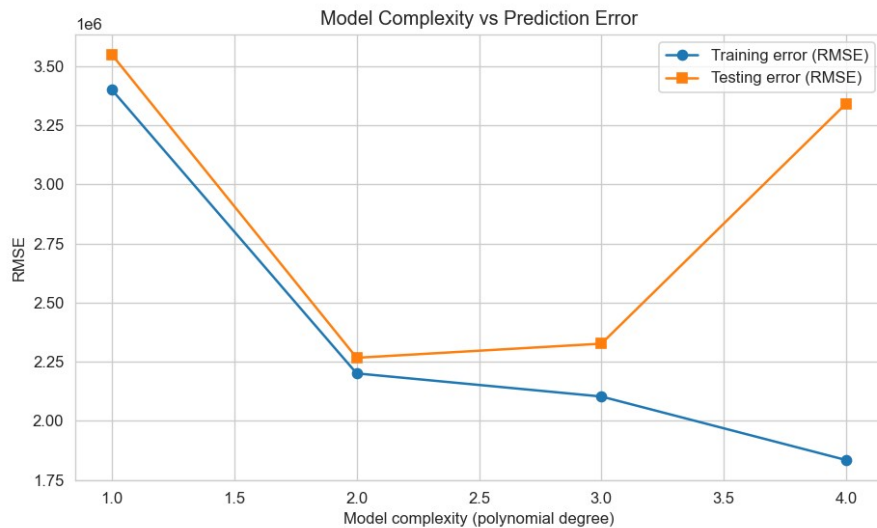


Figure 5: Model complexity (polynomial degree) vs training and testing RMSE

Degree	Train RMSE (₹)	Test RMSE (₹)	Gap	Diagnosis
1	3,400,099	3,548,650	small	Underfitting (high bias)
2	2,200,985	2,267,045	very small	Best fit
3	2,103,216	2,326,734	small	Starting to overfit
4	1,834,533	3,341,703	large	Overfitting (high variance)

OBSERVATION Training error keeps falling as the degree increases (3.40M → 1.83M), but test error falls until degree 2 and then rises sharply at degree 4 (3.34M). This U-shaped test curve is the classic signature of overfitting. Degree 1 is the underfitting side, degree 2 is the sweet spot.

11. Bias–Variance Trade-Off

DEFINITION **Bias** is the error from wrong or too-simple assumptions. **Variance** is the error from being too sensitive to the particular training sample. The **bias–variance trade-off** is the balance between the two: reducing one usually increases the other, and the best model minimises the total error.

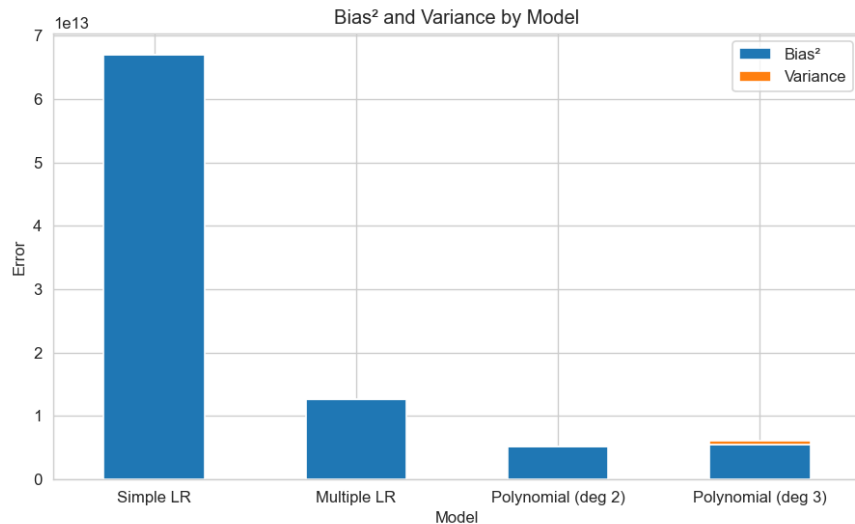
$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **High bias:** model misses the real pattern (underfitting). Example: predicting price from area only.
- **High variance:** the fitted model changes a lot if the training data changes (overfitting).
- **Irreducible error:** noise in the data that no model can remove.

Measured with 100 bootstrap resamples (test set)

Model	Bias ² (×10 ¹²)	Variance (×10 ¹²)	Train RMSE (₹)	Test RMSE (₹)
Simple LR	66,937	36	8,104,428	8,184,697
Multiple LR	12,597	42	3,400,099	3,548,650

Model	Bias ² ($\times 10^{12}$)	Variance ($\times 10^{12}$)	Train RMSE (₹)	Test RMSE (₹)
Polynomial (deg 2)	5,135	110	2,200,985	2,267,045
Polynomial (deg 3)	5,414	603	2,103,216	2,326,734

Figure 6: Bias² and variance for each model

OBSERVATION Simple LR has huge bias and tiny variance, so it underfits. Multiple LR cuts bias by 5 times. Polynomial degree 2 lowers bias again, while variance rises only a little. In degree 3, variance jumps about 5.5 times (110 $\rightarrow$ 603) but bias does not improve, so total error goes up. Degree 2 therefore gives the best balance.

Cross-validation check

5-fold cross-validation splits the data into 5 parts, trains on 4 and tests on 1, repeated 5 times. It gives a more reliable score than one split.

Model	CV R ² (mean)	CV R ² (std)
Simple LR	0.569	0.013
Multiple LR	0.923	0.004
Polynomial (deg 2)	0.967	0.003
Polynomial (deg 3)	0.964	0.004

The small standard deviations show the scores are stable across folds, and degree 2 is again the highest.

12. Gradient Descent Optimisation

DEFINITION Gradient Descent is an iterative optimisation algorithm that finds the model parameters (θ) that **minimise the cost function** (MSE) by repeatedly moving in the direction opposite to the gradient (the steepest downhill direction).

Analogy: you are on a foggy hill and want to reach the valley. At each step you feel the slope under your feet and step downhill. The step size is the learning rate.

$$\theta := \theta - \alpha \cdot \partial J / \partial \theta$$

$$J(\theta) = (1/m) \sum (\theta^T x_i - y_i)^2 \quad \text{Gradient} = (2/m) \cdot X^T (X\theta - y)$$

- **α (learning rate):** step size. Too small means very slow; too large means it overshoots and may diverge.
- **Epoch:** one full pass over the training data.
- **Convergence:** the point where the cost stops decreasing.
- **Feature scaling:** we standardise X and y first, otherwise features with big values (area, lot size) dominate and convergence becomes very slow.

Types of Gradient Descent

Type	Samples per update	Behaviour	Best used when
Batch GD	All training samples	Smooth, stable curve; one update per epoch	Small / medium data
Stochastic GD (SGD)	1 sample	Very noisy, zig-zag; many updates per epoch	Huge data, online learning
Mini-Batch GD	Small group (we used 32)	Fairly smooth and fast; the practical default	Most cases, deep learning

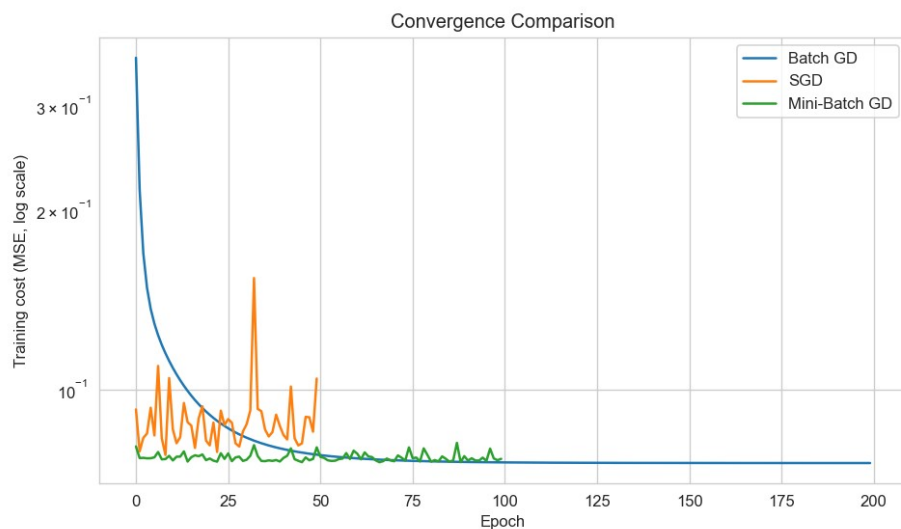


Figure 7: Convergence of Batch GD, SGD and Mini-Batch GD (log scale)

Method	Epochs	Final train cost	Time (s)	Test RMSE (₹)	Test R ²
Batch GD	200	0.0753	0.038	3,549,154	0.918
SGD	50	0.1044	1.961	4,275,429	0.881
Mini-Batch GD	100	0.0765	0.289	3,589,179	0.916
sklearn Multiple LR (closed form)	-	-	-	3,548,650	0.918

OBSERVATION Batch GD falls smoothly and its RMSE (3,549,154) is almost identical to the exact closed-form answer (3,548,650), which confirms our from-scratch code is correct. Mini-Batch drops fastest in the first epochs, then wiggles slightly, and ends very close (3,589,179). SGD is noisy with large spikes (learning rate 0.01 with one sample per step) and stops with the highest cost; it was also the slowest here, 1.96 s versus 0.04 s, because it runs a Python loop for every single sample. With a smaller learning rate or a decaying one, SGD would settle closer.

13. Final Model Comparison

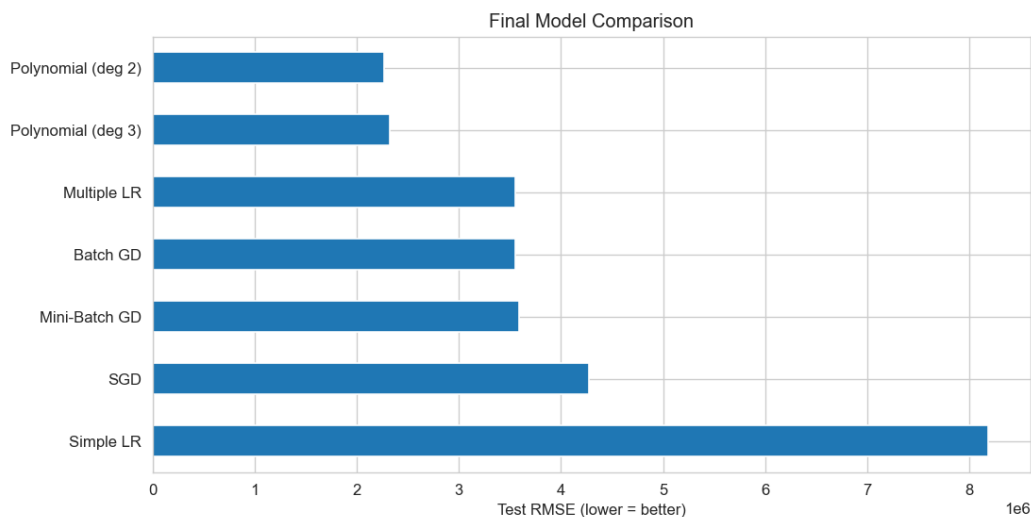


Figure 8: Test RMSE of all models (shorter bar = better)

Rank	Model	Test RMSE (₹)	R ²	Adj. R ²
1	Polynomial (degree 2)	2,267,045	0.966	0.964
2	Polynomial (degree 3)	2,326,734	0.965	0.946
3	Multiple LR	3,548,650	0.918	0.917
4	Batch GD	3,549,154	0.918	0.917
5	Mini-Batch GD	3,589,179	0.916	0.915
6	SGD	4,275,429	0.881	0.879
7	Simple LR	8,184,697	0.563	0.562

14. Conclusion and Business Interpretation

- **Best model:** Polynomial Regression (degree 2) has the lowest error (RMSE ≈ ₹22.7 lakh, about 10% of the average price) and the highest R² (0.966). It also has the best cross-validation score, so it is not a lucky split.
- **Why:** it captures interactions between features, and it sits at the bottom of the U-shaped test-error curve, so bias and variance are balanced.

- **Impact of gradient descent:** all three variants reach nearly the same solution as the exact formula. Mini-Batch gives the best speed and stability balance, Batch GD the smoothest path, and SGD the noisiest.
- **Overfitting / underfitting:** Simple LR underfits (R^2 0.56); polynomial degree 4 overfits (train RMSE 1.83M vs test 3.34M). Degree 2 fits well.
- **Business use:** the firm can price new listings quickly, find under-priced and over-priced houses, and see the main price drivers: **location score, area and number of rooms increase price, while age and distance from the city reduce it.** Predictions should support, not replace, expert judgement, because a typical error of about ₹23 lakh remains and very expensive houses are predicted less accurately.

15. Quick Glossary

Term	Definition
Feature	An input variable used for prediction.
Target / Label	The output value the model should predict.
Parameter (weight)	A number the model learns from data (β or θ).
Hyperparameter	A setting chosen by us before training (learning rate, degree, batch size).
Residual	Actual value minus predicted value.
Cost / Loss function	A number that measures how wrong the model is (MSE).
Generalisation	How well a model performs on new, unseen data.
Regularisation	A penalty on large coefficients (Ridge, Lasso) that reduces overfitting.
Cross-validation	Repeated train/test splitting to get a stable performance estimate.
Standardisation	Rescaling a feature to mean 0 and standard deviation 1.
Outlier	A data point far away from the others.